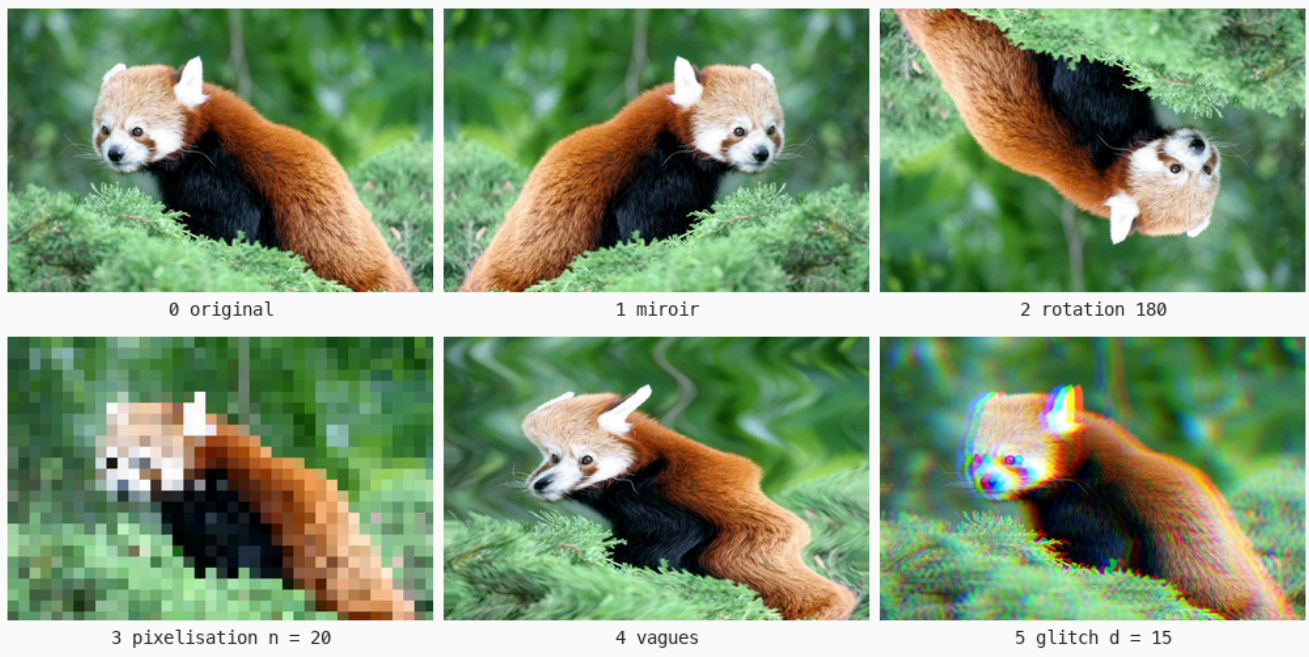


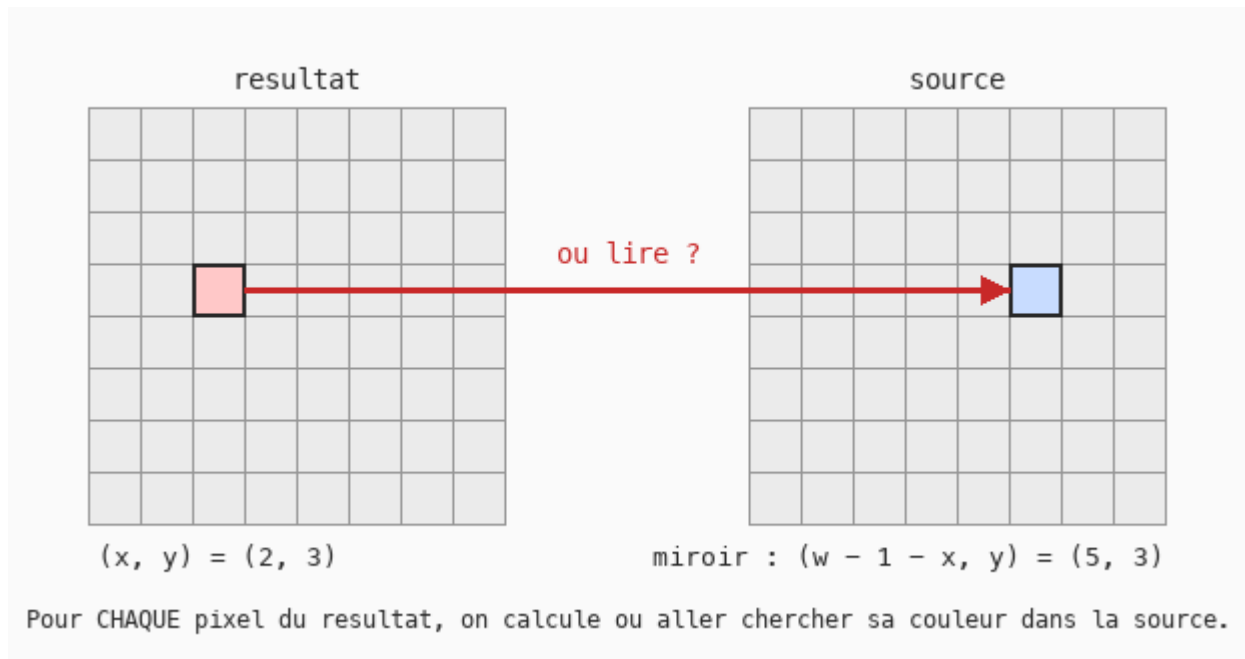
Cours 14 — Filtres géométriques : lire ailleurs

Avant : cours 11 (double boucle source / résultat), cours 13 (`sin` et le temps). **Comment travailler** : toujours dans `ofApp.h` et `ofApp.cpp` , par versions successives. `ofApp14.h` / `ofApp14.cpp` : la référence téléchargeable de l'état final. `pandaroux.jpg` va dans `bin/data` .

Au cours 11, chaque pixel changeait de **couleur** mais restait à sa place. Ici c'est l'inverse : la couleur ne change pas, c'est la **position de lecture** qui change. Miroir, pixelisation, vagues, glitch — et une façon de penser qui sert jusqu'aux shaders.



■ 1. Lire à l'envers



L'intuition dit : « prends chaque pixel de la source et déplace-le ». Ça ne marche pas — certains pixels du résultat recevraient deux couleurs, d'autres aucune : des trous.

On raisonne toujours **depuis le résultat** : pour chaque pixel (x, y) du résultat, on calcule à quel endroit de la source aller **chercher** sa couleur :

```
resultat(x, y) = source(f(x, y))
```

Comme la boucle visite chaque pixel du résultat exactement une fois, l'image est toujours pleine. Cette façon de penser — du pixel de sortie vers l'entrée — est celle de **tous** les effets d'image, jusqu'aux shaders.

■ 2. Étape 1 — la machine, et une lecture blindée

Même mise en place qu'au cours 11 (`source`, `resultat`, `effet`, `parametre`, plus `float t { 0 };`). La nouveauté est une fonction de lecture qui ne sort **jamais** de l'image (annonce dans le `.h` : `ofColor lire(float x, float y);`):

```
ofColor ofApp::lire(float x, float y) {
    int px = ofClamp((int)x, 0, source.getWidth() - 1);
    int py = ofClamp((int)y, 0, source.getHeight() - 1);
    return source.getColor(px, py);
}
```

Un miroir ou une vague peut demander le pixel -5 ou le pixel 800 : on borne systématiquement (cours 10), une fois pour toutes, dans une fonction. Le `(int)x` **convertit** le `float` en entier en coupant la partie après la virgule — les coordonnées d'un pixel sont entières.

La machine, dans `update()`, commence par l'identité :

```

parametre = mouseX / (float)ofGetWidth();
t = ofGetElapsedTimef();
int w = source.getWidth();
int h = source.getHeight();

for (int y = 0; y < h; y++) {
    for (int x = 0; x < w; x++) {
        ofColor c = lire(x, y);          // identité : lire au même endroit
        resultat.setColor(x, y, c);
    }
}
resultat.update();

```

L'image s'affiche, copiée pixel par pixel.

Essaie : remplace par `lire(x, y + 50)` — l'image **monte** de 50 pixels. Lire plus bas fait remonter l'image : c'est la lecture à l'envers, ressens-la bien. Et regarde le bord : le dernier pixel s'étire (l'effet du `ofClamp`).

■ 3. Étape 2 — miroir et rotation

Installe le `switch` du cours 11 dans la boucle, avec les deux premières formules :

```

switch (effet) {
case 1:                                // miroir horizontal
    c = lire(w - 1 - x, y);
    break;

case 2:                                // rotation de 180 degrés
    c = lire(w - 1 - x, h - 1 - y);
    break;

default:
    c = lire(x, y);
}

```

(Et le `keyPressed` du cours 11 pour choisir l'effet.) Le miroir : la colonne `x` du résultat vient de la colonne **opposée** de la source. La rotation à 180° : les deux axes retournés. Note le `break;` à la fin de chaque `case` : ici les branches ne font pas `return` (on est au milieu d'une boucle), c'est lui qui sort du `switch`.

Essaie : écris le miroir **vertical**. Puis prédis : que fait `lire(w - 1 - x, h - 1 - y)` appliqué deux fois de suite ?

■ 4. Étape 3 — la pixelisation

```
case 3: {  
    int n = 2 + parametre * 40;  
    c = lire(x / n * n, y / n * n);  
    break;  
}
```

La **division entière** du cours 01, devenue un outil : $x / n * n$ n'est pas x ! Avec $n = 20$, $137 / 20 * 20 = 120$ — tous les x de 120 à 139 lisent la colonne 120. L'image se lit par cases de $n \times n$: de gros pixels, réglés à la souris.

Essaie : fixe n à 2, 10, puis 60 — à partir de quand l'image devient-elle méconnaissable ? (Compare avec la grille du cours 12 : deux chemins vers le même effet.)

■ 5. Étape 4 — les vagues

```
case 4: {  
    float amplitude = parametre * 40;  
    float dx = amplitude * sin(y / 20.0f + t * 3);  
    c = lire(x + dx, y);  
    break;  
}
```

Chaque **ligne** est décalée d'un sinus : $\sin(y / 20.0f)$ oscille le long de la hauteur, $+ t * 3$ fait glisser l'ondulation avec le temps, amplitude la dose en pixels. Le $\sin$ du cours 13, appliqué à une position de lecture — l'image ondule comme un drapeau.

Essaie : des vagues **verticales** (décaler y selon x). Puis les deux à la fois.

■ 6. Étape 5 — le glitch : un canal, un endroit

```
case 5: {  
    float d = parametre * 30;  
    ofColor gauche = lire(x - d, y);  
    ofColor centre = lire(x, y);  
    ofColor droite = lire(x + d, y);  
    c = ofColor(gauche.r, centre.g, droite.b);  
    break;  
}
```

Rien n'oblige les trois canaux à venir du même pixel. Le rouge est lu un peu à gauche, le bleu un peu à droite : les contours se dédoublent en franges colorées — l'**aberration chromatique**, très utilisée dans les jeux pour un effet « écran abîmé ». Trois lectures, une recomposition avec les `.r`, `.g`, `.b` de la struct `ofColor` (cours 09).

Essaie : pousse `d` très haut. Puis décale un **seul** canal — lequel se voit le plus ?

■ Exercices

1. **Lire avant de lancer** — que fait `c = lire(x + 50, y);` : l'image se décale-t-elle vers la gauche ou vers la droite ? Et `lire(x * 2, y)` : zoom avant, ou arrière ? Réponds, puis vérifie — c'est LE test de la lecture à l'envers.
2. **Le kaléidoscope** — la moitié droite de l'image est le reflet de la moitié gauche : un `if (x > w / 2)` dans la formule. Puis fais pareil verticalement, pour un quadruple reflet.
3. **Le glitch de lignes** — des lignes entières décalées d'une valeur `ofRandom(-30, 30)` — avec `ofSeedRandom` pour que ça ne scintille pas (cours 03)... ou sans, pour que ça scintille exprès. Le vrai look « VHS abîmée ».
4. **Le zoom (plus costaud)** — `c = lire(x * 0.5f + w / 4, y * 0.5f + h / 4);` : un zoom $\times 2$ centré. D'où viennent le `0.5f` et le `w / 4` ? Explique chaque nombre, puis écris le zoom **arrière**.

La loupe et le tourbillon, eux, demandent de passer en coordonnées polaires : on y revient à la fin du cours 16.

■ Ce qu'il faut retenir

- **Lire à l'envers** : pour chaque pixel du résultat, calculer où lire dans la source — jamais l'inverse. Le réflexe de tous les effets d'image.
- Une fonction `lire(x, y)` bornée par `ofClamp` : plus jamais hors de l'image (le bord s'étire, c'est le choix simple).
- `(int)x` : convertir un `float` en entier en coupant la virgule.
- La division entière comme outil : `x / n * n` aligne sur des cases de taille `n`.
- Miroir = coordonnée opposée ; vagues = `sin` sur la position de lecture ; glitch = un canal, un endroit.
- Dans un `switch` au milieu d'une boucle : `break;` pour sortir de la branche.